0377 · ADMINISTRACIÓN DE SISTEMAS GESTORES DE BASES DE DATOS

Instalación y aseguramiento de los SGBD

¿En qué se parecen y en qué se diferencian la instalación y el aseguramiento inicial de SQL Server, PostgreSQL y MongoDB?

EQUIPO	**DB Hunters** · Pedro Hosannah · Antonio Moga · Hugo López · Daniel Madrigal
LÍNEA	L4 — Instalación y aseguramiento
SGBD COMPARADOS	Microsoft SQL Server (comercial) · PostgreSQL (libre) · MongoDB (NoSQL documental)
PRODUCT OWNER	Raúl Albiol Salguero — se mantiene durante los tres sprints
ROLES · SPRINT 1	SM: Daniel Madrigal · Desarrollo: Pedro Hosannah, Antonio Moga, Hugo López
ROLES · SPRINT 2	SM: Pedro Hosannah · Desarrollo: Daniel Madrigal, Antonio Moga, Hugo López
ROLES · SPRINT 3	SM: Antonio Moga · Desarrollo: Pedro Hosannah, Daniel Madrigal, Hugo López
PERIODO	25 de septiembre – 7 de octubre de 2026 · tres sprints
MÓDULO	Unidad 1 · Instalación del SGBD · 2.º CFGS ASIR · Curso 2026-2027

Entrega del Sprint 3 · 7 de octubre de 2026 · Informe de investigación, Guía de trabajo y organización y Google Sheets Scrum

— Índice

1 Introducción 3

2 Marco común: qué es un SGBD 3

2.1 Funciones y componentes 4

2.2 La arquitectura ANSI/SPARC 4

2.3 Arquitectura cliente-servidor y tipos de SGBD 4

3 Metodología de la investigación 5

4 Desarrollo de la línea L4 6

4.1 Requisitos previos 6

4.2 Paquetes, instaladores y métodos de despliegue 6

4.3 Parámetros relevantes de la configuración inicial 7

4.4 Qué viene abierto de fábrica y cómo se cierra 8

5 Comparativa de los tres motores 9

5.1 Fichas básicas 9

5.2 Tabla comparativa 9

6 Conclusiones y recomendaciones 11

— Referencias 12

A Anexo A · Evidencias de la verificación práctica 13

B Anexo B · Declaración de uso de IA y registro Scrum 15

> **Cómo leer este informe**
>
> Las afirmaciones tomadas de una fuente llevan su referencia numerada[n], que remite a la lista final. Se distingue explícitamente entre lo que documenta el fabricante, lo que hemos comprobado nosotros en la máquina de pruebas y lo que es valoración propia del equipo; esta última aparece siempre señalada como tal.

1 Introducción

> *Un motor recién instalado no es un sistema seguro. Es un sistema con la configuración que trae de fábrica, y esa configuración está pensada para que el servicio arranque, no para que aguante.*

La pregunta de nuestra línea es **en qué se parecen y en qué se diferencian la instalación y el aseguramiento inicial de los SGBD más usados**. Dicho de otra forma: cuando el administrador termina de instalar y el servicio ya responde, ¿qué ha quedado expuesto sin que nadie lo decidiera? ¿Qué eligió el instalador por él? ¿Y qué le queda por cerrar a mano?

Nos pareció una pregunta que merecía la pena por dos motivos. El primero es que la instalación es el único momento del ciclo de vida de un SGBD en el que se fijan parámetros que luego cuesta mucho deshacer. Dónde viven los ficheros de datos, con qué codificación se guarda el texto o con qué modo de autenticación arranca el servidor se decide una sola vez, en la pantalla del instalador o en la línea de comandos. Cambiarlo más tarde obliga a migrar los datos o a reinstalar.

El segundo motivo es más práctico. Los incidentes de seguridad con bases de datos no suelen venir de fallos sofisticados del motor, sino de instancias que se quedaron escuchando en todas las interfaces de red, sin autenticación, o con la cuenta de administrador accesible desde fuera. Saber qué viene abierto de fábrica es algo que un técnico de sistemas debería tener claro antes de poner un servidor en producción.

Para responder comparamos tres motores de familias distintas, cumpliendo la norma del encargo de incluir uno comercial, uno libre y uno NoSQL:

- **Microsoft SQL Server**, relacional comercial, con instalador gráfico e integración con la seguridad de Windows.
- **PostgreSQL**, relacional libre, que se instala desde los repositorios de la distribución y tiene el concepto propio de clúster de datos.
- **MongoDB**, NoSQL documental, que se instala con el metapaquete oficial del fabricante y cuyo control de acceso no está activo de inicio.

Los tres cubren entornos diferentes: Windows, Linux nativo y contenedores. Eso nos permitía ver si las diferencias de aseguramiento son del motor o del sistema operativo que lo aloja. Como verificación práctica instalamos y probamos PostgreSQL 18.6 sobre UbuntuEscritorio26, en una máquina virtual de Oracle VirtualBox, y dejamos documentado con capturas propias el arranque del servicio, la conexión, la consulta al catálogo y dónde quedan los logs (Anexo A).

2 Marco común: qué es un SGBD

Un **Sistema Gestor de Bases de Datos (SGBD)** es el software que permite trabajar con una base de datos y que hace de intermediario entre los usuarios o las aplicaciones y los datos almacenados. Conviene no mezclar los dos términos. La *base de datos* es el conjunto de datos relacionados y su descripción. El *SGBD* es el programa que los almacena, consulta, modifica y administra de forma organizada.[1][2]

Lo que define a un SGBD es que separa la estructura lógica de los datos de su almacenamiento físico. Gracias a esa separación, un administrador puede mover ficheros, cambiar discos, reorganizar índices o ajustar la memoria del servicio sin tocar el código de las aplicaciones que consultan esos datos.[1]

2.1 Funciones y componentes

Estas son las funciones principales que presta cualquier SGBD, sea del fabricante que sea:

- **Gestión del diccionario de datos.** Guarda los metadatos, es decir, las definiciones de estructuras, relaciones, restricciones y usuarios. Actúa como guía estructural del sistema y es el catálogo que consultamos en la prueba práctica.
- **Definición y manipulación de datos.** Ofrece lenguajes normalizados para crear la base de datos y para consultar o modificar su contenido de manera predecible.
- **Seguridad y control de acceso.** Administra la autenticación y los permisos para que cada usuario o aplicación llegue solo a lo que tiene autorizado. Es la función sobre la que gira toda nuestra línea.
- **Control de concurrencia.** Gestiona el acceso simultáneo de varios usuarios a los mismos datos mediante transacciones, evitando bloqueos e inconsistencias.
- **Integridad y recuperación.** Mantiene el log de transacciones, que garantiza la integridad y permite recuperar el servicio tras un fallo de hardware o una parada brusca.
- **Gestión del almacenamiento y la memoria.** Habla con el sistema operativo para administrar la entrada y salida en disco y los buffers de memoria del servicio.

A esas funciones les corresponden componentes que se reconocen en cualquier motor: el *motor de almacenamiento*, que escribe y lee los ficheros físicos; el *procesador de consultas*, que analiza y optimiza las peticiones; el *gestor de transacciones*, que aplica las propiedades ACID; y el *diccionario de datos*, que describe todo lo anterior. Visto así, instalar un SGBD consiste en colocar esos componentes en disco y decidir con qué parámetros arrancan.

2.2 La arquitectura ANSI/SPARC

El modelo ANSI/SPARC describe un SGBD en tres niveles de abstracción y explica por qué es posible separar la lógica del almacenamiento:

- **Nivel externo.** Es lo que ve cada usuario o aplicación: vistas parciales de los datos adaptadas a lo que necesita y a los permisos que tiene. Un mismo dato puede presentarse de formas distintas a usuarios distintos.
- **Nivel conceptual.** Es el esquema lógico global. Define qué entidades existen, qué atributos tienen y qué restricciones de integridad se aplican, sin decir nada de cómo se guardan.
- **Nivel interno.** Es la representación física: ficheros, bloques, índices y rutas en disco. Aquí es donde encaja el *directorio de datos* del que hablamos en el apartado 4.

De esta estructura salen las dos independencias que dan valor al modelo. La **independencia física** permite cambiar el nivel interno, por ejemplo mover el directorio de datos a otro disco, sin alterar el esquema conceptual. La **independencia lógica** permite modificar el esquema conceptual sin reescribir las vistas de los usuarios. Para nuestra línea el matiz importante es que la instalación trabaja sobre el nivel interno: elegir dónde viven los ficheros y con qué codificación se escriben es una decisión de nivel interno que los otros dos niveles heredan.

2.3 Arquitectura cliente-servidor y tipos de SGBD

Los tres motores que estudiamos funcionan en **arquitectura cliente-servidor**. Un proceso servidor (`sqlservr`, `postgres`, `mongod`) se ejecuta de forma permanente, queda a la escucha en un puerto de red y atiende las peticiones de procesos cliente (`sqlcmd` o SSMS, `psql`, `mongosh`) que pueden estar en la misma máquina o en otra. Esta arquitectura es la razón de ser de nuestra línea: todo lo que el motor expone a la red, desde el puerto hasta el método de autenticación, es consecuencia directa de ser un servidor, y es justo lo que la instalación deja configurado de una forma o de otra. Frente a este modelo, los motores *em-*

bebidos como SQLite se ejecutan dentro del propio proceso de la aplicación y no abren ningún puerto, así que su aseguramiento se reduce a los permisos del fichero.

En cuanto a la clasificación, hoy los SGBD se agrupan en relacionales y objeto-relacionales (SQL Server, PostgreSQL, Oracle, MySQL y MariaDB), NoSQL en sus variantes documental, clave-valor, columnar y de grafos (MongoDB, Redis, Cassandra, Neo4j), vectoriales para cargas de IA (pgvector, Milvus) y servicios gestionados en la nube o DBaaS (Amazon RDS, Azure Database, Cloud SQL). Nuestra comparativa toma un representante de las tres primeras familias que admiten instalación autogestionada.

3 Metodología de la investigación

Fuentes. Hemos trabajado con 24 fuentes distintas. De ellas, 20 son documentación oficial de los tres fabricantes: Microsoft Learn, el PostgreSQL Global Development Group y MongoDB Inc. El mínimo que pedía el encargo era de seis fuentes y tres documentaciones oficiales. Las cuatro restantes son material de referencia general sobre SGBD, la imagen oficial de PostgreSQL en Docker y el benchmark de seguridad del Center for Internet Security. Este último lo incluimos a propósito, porque queríamos tener una fuente independiente del fabricante con la que contrastar las recomendaciones de seguridad de Microsoft.[1][2][15][19]

Criterios de la comparativa. Antes de empezar a investigar fijamos una rejilla común de once criterios, para que los tres motores se estudiaran exactamente igual: requisitos, método de instalación, paquetes o instaladores, sistema operativo o entorno, directorio de datos, puerto por defecto, juego de caracteres, usuario administrador o sistema de autenticación, configuración inicial, servicios y accesos habilitados por defecto, y medidas necesarias para asegurar la instalación. Comprobamos que la rejilla valía para los tres antes de repartir el trabajo, y eso nos ahorró tener que rehacer apartados más tarde.

Contraste. Los datos en los que es fácil equivocarse, como puertos, rutas, codificaciones y comportamiento por defecto, los hemos confirmado en al menos dos fuentes independientes o directamente en la prueba práctica. Por ejemplo, el puerto 5432 y el valor por defecto de `listen_addresses` en PostgreSQL aparecen tanto en la documentación de administración del servidor como en la de parámetros de conexión. En cambio, la codificación UTF8 con locale `es_ES.UTF-8` no la sacamos de la documentación: la leímos en el catálogo de nuestra propia instancia.[16][18]

Verificación práctica. Instalamos PostgreSQL por línea de comandos sobre UbuntuEscritorio26, en una máquina virtual de Oracle VirtualBox, el 2 de octubre de 2026. La prueba cubre los cuatro mínimos que pedía el encargo (servicio arrancado, conexión establecida, consulta al diccionario de datos y localización de los ficheros de log) y además creamos una base de datos de prueba llamada `prueba_dbhunters` para confirmar que el catálogo refleja los cambios. Las capturas están en el Anexo A, comentadas y con su fecha.

Hecho frente a opinión. A lo largo del informe, lo que afirma el fabricante aparece atribuido («Microsoft recomienda…», «la documentación de MongoDB advierte…»), lo que comprobamos nosotros se señala como resultado de la prueba, y nuestras valoraciones van marcadas como tales en recuadros aparte.

Uso de IA. Hemos usado asistentes de IA generativa para localizar documentación, resumir apartados largos de las guías oficiales, traducir fragmentos en inglés y revisar la redacción. Ningún dato técnico ha entrado en el informe sin verificarlo después en la fuente oficial o en la máquina de pruebas. El detalle está en el Anexo B.

> **Limitación del estudio**
>
> Solo uno de los tres motores se ha instalado y probado de verdad. Los datos de SQL Server y MongoDB salen de documentación oficial contrastada, pero no de una verificación nuestra, así que no podemos descartar diferencias entre lo documentado y el comportamiento real de una instalación concreta. Lo decimos para que el lector pondere en consecuencia esas filas de la comparativa.

4 Desarrollo de la línea L4

> *Instalar los tres motores se parece mucho más de lo que esperábamos. Asegurarlos, en cambio, obliga a tocar puntos muy distintos de cada sistema.*

4.1 Requisitos previos

SQL Server es el único de los tres que publica requisitos de hardware cerrados. La versión 2025 pide un mínimo de 512 MB de memoria en la edición Express y 1 GB en el resto, aunque Microsoft recomienda 4 GB o más. A eso se suman 6 GB libres en disco y un procesador x64 de 1,4 GHz. Los sistemas soportados son Windows 10 o superior y Windows Server 2019 o superior, con .NET Framework 4.7.2, y el fabricante desaconseja expresamente instalarlo en un controlador de dominio. En Linux la cifra sube: la guía de Ubuntu pide al menos 2 GB de RAM.[3][14]

PostgreSQL hace justo lo contrario. Su documentación no fija requisitos mínimos de hardware en el apartado de instalación. Lo que detalla es el procedimiento para inicializar el clúster de datos con la utilidad `initdb`. La exigencia real la pone la distribución y la carga prevista, no el motor.[16]

MongoDB introduce un requisito que puede dejar fuera equipos antiguos. A partir de las versiones recientes el procesador tiene que soportar obligatoriamente el conjunto de instrucciones **AVX**, y un equipo sin AVX no arranca el servicio. La microarquitectura mínima es Haswell en la línea Intel Core (o Tiger Lake en Celeron y Pentium), Bulldozer o superior en AMD, y ARMv8.2-A o superior en ARM64. En memoria no impone un límite rígido, pero el motor de almacenamiento reserva un mínimo de 256 MB de caché interna y su fórmula por defecto toma el 50 % de la RAM disponible menos 1 GB. Para producción recomienda disco SSD en lugar de HDD y, en Linux, sistema de ficheros XFS.[21]

> **Valoración del equipo**
>
> La diferencia de estilo importa a quien tiene que planificar una instalación. SQL Server da una cifra que puedes llevar directamente a un pliego de requisitos. MongoDB da una condición de CPU que es binaria y conviene verificar *antes* de provisionar la máquina, porque no se arregla añadiendo memoria. PostgreSQL deja la decisión al administrador, lo cual es flexible pero obliga a dimensionar con criterio propio.

4.2 Paquetes, instaladores y métodos de despliegue

SQL Server admite dos vías en Windows. El asistente gráfico guía la elección de edición y licencia, la selección de características, el nombre de la instancia (por defecto `MSSQLSERVER`, o con nombre propio), las cuentas de servicio, la collation, el modo de autenticación, los administradores del sistema y los directorios de datos. La instalación desatendida por línea de comandos expone los mismos parámetros como argumentos: `/ACTION`, `/FEATURES`, `/INSTANCENAME`, `/SQLSVCACCOUNT`, `/SQLSYSADMINACCOUNTS`, `/SECURITYMODE`, `/SAPWD`, `/INSTALLSQLDATADIR`, `/TCPENABLED`, `/SQLCOLLATION` y `/IACCEPTSQLSERVERLICENSETERMS`. En Linux se instala el paquete `mssql-server` y se completa con `mssql-conf setup`, donde se eligen la edición y la contraseña de SA.[4][5][14]

PostgreSQL se instala en Debian y Ubuntu con APT, mediante los paquetes `postgresql` y `postgresql-contrib`, y el clúster de datos se prepara con `initdb`. En nuestra prueba el propio paquete de Ubuntu hizo esa inicialización y dejó el servicio habilitado sin que tuviéramos que intervenir. Como alternativa, la imagen oficial de Docker permite desplegar el motor pasando los parámetros de instalación como variables de entorno y guardando los datos en un volumen.[16][19]

MongoDB se instala en Linux con el gestor de paquetes del sistema usando el metapaquete oficial `mongodb-org`, que trae el motor, la consola interactiva y las utilidades asociadas. Hay un detalle fácil de pasar por alto: la documentación avisa de que el paquete `mongodb` incluido en los repositorios de Ubuntu no

está mantenido por MongoDB y entra en conflicto con `mongodb-org`. Si no se lee ese aviso, la instalación puede quedarse a medias.[20]

4.3 Parámetros relevantes de la configuración inicial

Directorio de datos

En SQL Server sobre Windows la ruta por defecto es `C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER\MSSQL\Data`, donde el identificador de instancia se forma con la versión y el nombre de la instancia, y se puede cambiar durante la instalación. En Linux los datos van a `/var/opt/mssql`. En PostgreSQL el directorio de datos se define al inicializar el clúster y, si se usa Docker, la persistencia se gestiona con un volumen montado en `/var/lib/postgresql/data`. En nuestra instalación nativa los logs quedaron en `/var/log/postgresql/`, como se ve en la prueba. En MongoDB la instalación por APT crea por defecto el directorio de datos `/var/lib/mongodb` y el de logs `/var/log/mongodb`.[6][14][16][19][20]

Puerto y exposición de red

Los tres motores escuchan en un puerto fijo y conocido. SQL Server usa TCP 1433 para la instancia por defecto, al que se suman UDP 1434 del servicio SQL Server Browser y puertos dinámicos para las instancias con nombre. PostgreSQL usa TCP 5432 y `mongod` el TCP 27017. Que el puerto sea conocido no es un problema en sí, pero sí lo convierte en objetivo previsible de escaneo. La cuestión importante no es el número de puerto, sino en qué interfaz se escucha y quién puede llegar hasta ella, que es de lo que trata el apartado 4.4.[7][18][22]

Juego de caracteres y codificación

Aquí aparece una de las diferencias conceptuales más claras entre los tres. En **SQL Server** la collation del servidor se elige durante la instalación a partir de la configuración regional del sistema operativo, y desde la versión 2019 existen collations UTF-8. En **PostgreSQL** la codificación y el locale se asignan al inicializar el clúster; en nuestra prueba, al consultar el catálogo, la instancia aparecía con codificación UTF8 y locale `es_ES.UTF-8` heredados del sistema. En **MongoDB** la pregunta ni siquiera se plantea: la información se guarda en formato BSON, cuya especificación obliga a que todas las cadenas estén en UTF-8, no hay parámetro de configuración para cambiarlo, y son los drivers de cada lenguaje los que convierten el texto antes de enviarlo al motor.[13][16][23]

Usuario administrador y autenticación

La cuenta administrativa de **SQL Server** es `sa`. Si se instala con autenticación de Windows, esa cuenta queda deshabilitada y con una contraseña aleatoria, y Microsoft recomienda no habilitarla salvo necesidad y, si se habilita, ponerle una contraseña robusta. El motor se ejecuta además con la cuenta virtual `NT SERVICE\MSSQLSERVER`, y el fabricante insiste en aplicar el principio de mínimo privilegio a las cuentas de servicio.[9][10]

En **PostgreSQL** el administrador inicial es el usuario `postgres`, que en una instalación nativa sobre Linux se corresponde con un usuario del sistema. De ahí que la conexión de nuestra prueba se hiciera con `sudo -u postgres psql`. En Docker, el usuario y su contraseña se definen con las variables `POSTGRES_USER` y `POSTGRES_PASSWORD`.[16][19]

En **MongoDB** no hay administrador inicial. La documentación establece que hay que crear un usuario con privilegios administrativos *antes* de activar el control de acceso, porque al poner `security.authorization: enabled` el motor pasa a aplicar control de acceso basado en roles y cada usuario solo puede ejecutar las acciones que tenga concedidas.[22][24]

4.4 Qué viene abierto de fábrica y cómo se cierra

Este es el núcleo de la pregunta de investigación, y es donde los tres motores se separan de verdad.

SQL Server llega con una superficie de ataque bastante reducida. Shared Memory está habilitado en todas las ediciones, pero TCP/IP solo lo está en Enterprise, Standard y Evaluation, y queda deshabilitado en Developer y Express. Named Pipes viene deshabilitado para conexiones de red. El servicio SQL Server Browser también, y en la edición Express el Agente igual. La función `xp_cmdshell`, que permite ejecutar comandos del sistema operativo desde el motor, está deshabilitada en instalaciones nuevas. Lo que el instalador *no* hace es abrir los puertos en el firewall de Windows: esas reglas hay que crearlas a mano. [7][8][9][12]

El resto de recomendaciones de aseguramiento de Microsoft se aplican antes y después de instalar: usar firewall, aislar los servicios con cuentas de pocos privilegios, emplear NTFS o ReFS, desactivar NetBIOS y SMB en servidores expuestos y preferir autenticación de Windows con contraseñas robustas para `sa`. Todo eso puede contrastarse además con el benchmark independiente del CIS.[11][15]

PostgreSQL cierra la puerta con dos cerrojos distintos, y conviene no confundirlos. El primero es `listen_addresses`, que por defecto vale `localhost`: mientras no se cambie, las conexiones TCP/IP se limitan a la propia máquina. El segundo es `pg_hba.conf`, que controla qué clientes pueden autenticarse, desde qué direcciones y con qué método. Abrir solo el primero sin revisar el segundo, o al revés, es el error clásico. Un administrador puede exponer el servicio a la red y seguir pensando que está protegido porque «no ha tocado los permisos». Para las conexiones con contraseña, la documentación recomienda métodos robustos como SCRAM-SHA-256.[17][18]

MongoDB es el caso que más atención pide. Por defecto `net.bindIp` vale `localhost` y `net.bindIpAll` está desactivado, así que el motor escucha solo en el equipo local. Pero `security.authorization` también está **desactivado** por defecto, lo que significa que cualquiera que alcance el puerto 27017 opera sin autenticarse. Mientras la escucha esté limitada a `localhost` el riesgo está contenido. El problema es que las dos opciones se tocan en el mismo fichero `/etc/mongod.conf`, y basta cambiar una sin cambiar la otra para publicar una base de datos sin credenciales. La propia documentación avisa de que, antes de vincular MongoDB a una dirección accesible desde la red, hay que habilitar la autenticación y asegurar la infraestructura. El checklist de seguridad del fabricante completa el procedimiento: crear el usuario administrativo, activar el control de acceso basado en roles, limitar con firewall el tráfico al puerto 27017 a direcciones de confianza y configurar certificados TLS/SSL para cifrar las comunicaciones en tránsito.[20][22][24]

> **Valoración del equipo**
>
> Los tres motores comparten el mismo acierto de diseño, que es no escuchar en todas las interfaces de red nada más instalarse. Pero reparten la responsabilidad de forma muy distinta. SQL Server deshabilita funciones peligrosas por su cuenta y deja al administrador una tarea concreta y visible, el firewall. PostgreSQL obliga a entender dos ficheros que actúan en capas independientes. MongoDB es el único que arranca sin autenticación, y por tanto el único en el que un solo parámetro mal puesto convierte una instancia correcta en una base de datos pública. A nuestro juicio, si alguien despliega sin leer la documentación, el orden de riesgo de mayor a menor es MongoDB, PostgreSQL y SQL Server.

5 Comparativa de los tres motores

5.1 Fichas básicas

Microsoft SQL Server

TIPO	Relacional
LICENCIA	Comercial; edición Express gratuita
VERSIÓN ESTUDIADA	SQL Server 2025 (17.x), con referencias comunes a 2017–2025
PUERTO	TCP 1433 (instancia por defecto) · UDP 1434 (Browser)
ADMINISTRACIÓN	Asistente de instalación, SQL Server Configuration Manager, SSMS, `sqlcmd`, `mssql-conf` en Linux

PostgreSQL

TIPO	Objeto-relacional
LICENCIA	Libre (licencia PostgreSQL)
VERSIÓN ESTUDIADA	PostgreSQL 18 · versión verificada en la prueba: **18.6**
PUERTO	TCP 5432
ADMINISTRACIÓN	`initdb`, `psql`, `postgresql.conf`, `pg_hba.conf`, imagen oficial de Docker

MongoDB

TIPO	NoSQL documental (BSON)
LICENCIA	SSPL y comercial
VERSIÓN ESTUDIADA	MongoDB 8.0 Community Edition
PUERTO	TCP 27017
ADMINISTRACIÓN	`mongosh`, `/etc/mongod.conf`, `systemctl`, metapaquete `mongodb-org`

5.2 Tabla comparativa

CRITERIO	SQL SERVER	POSTGRESQL	MONGODB
Requisitos	512 MB RAM (Express) / 1 GB resto; 4 GB recomendados. 6 GB libres, CPU x64 1,4 GHz. Windows 10+/Server 2019+ y .NET 4.7.2[3]	La documentación no fija mínimos de hardware; la preparación se centra en el clúster vía `initdb`[16]	CPU con AVX obligatorio; microarquitectura mínima por plataforma. Caché interna mínima de 256 MB; SSD y XFS recomendados[21]
Método de instalación	Asistente gráfico o instalación desatendida por comandos. En Linux, `mssql-conf setup`[4][5][14]	APT en Ubuntu e inicialización del clúster con `initdb`; alternativa en Docker[16][19]	Gestor de paquetes del sistema con el metapaquete oficial `mongodb-org`[20]

CRITERIO	SQL SERVER	POSTGRESQL	MONGODB
Paquetes o instaladores	Setup de SQL Server en Windows; paquete `mssql-server` en Linux[14]	`postgresql` y `postgresql-contrib`; imagen oficial de Docker[16][19]	`mongodb-org`: motor, consola y utilidades. El paquete `mongodb` de Ubuntu entra en conflicto[20]
SO o entorno	Windows como entorno principal; Ubuntu 22.04 y 24.04 documentados[3][14]	Multiplataforma. Verificado en UbuntuEscritorio26 sobre VirtualBox; también Docker	SO compatible de 64 bits; ciclo de vida con el gestor de servicios nativo[20]
Directorio de datos	Windows: … `\MSSQL17.MSSQLSERVER\MSSQL\Data`, modificable en la instalación. Linux: `/var/opt/mssql`[6][14]	Se define al inicializar el clúster; en Docker, volumen en `/var/lib/postgresql/data`. Logs verificados en `/var/log/postgresql/`[16][19]	`/var/lib/mongodb`; logs en `/var/log/mongodb`[20]
Puerto por defecto	TCP 1433; UDP 1434 para Browser; puertos dinámicos en instancias con nombre[7]	TCP 5432[18]	TCP 27017[22]
Juego de caracteres	Collation elegida en la instalación según la configuración regional; collations UTF-8 desde 2019[13]	Definido al inicializar. Verificado en la prueba: UTF8 con locale `es_ES.UTF-8`	BSON obliga a UTF-8 en todas las cadenas; no es configurable[23]
Administrador y autenticación	`sa`; deshabilitada con autenticación de Windows. Se recomienda no habilitarla salvo necesidad[10]	Usuario `postgres`; en Docker, `POSTGRES_USER` y `POSTGRES_PASSWORD`[16][19]	Sin administrador inicial: hay que crearlo antes de activar el control de acceso[24]
Configuración inicial	Instancia, cuentas de servicio, collation, modo de autenticación, administradores y directorios de datos[4]	`initdb` para el clúster y `pg_hba.conf` para los accesos[16][17]	`/etc/mongod.conf`; los cambios requieren reiniciar la instancia[20][22]
Accesos por defecto	Shared Memory activo; TCP/IP según edición; Named Pipes y Browser deshabilitados[8][9]	`listen_addresses = localhost`; `pg_hba.conf` controla la autenticación de clientes[17][18]	Escucha solo en localhost; `bindIpAll` y **`security.authorization` desactivados**[22]
Cómo se cierra	Crear reglas de firewall a mano, mínimo privilegio, NTFS/ReFS, limitar NetBIOS/SMB, autenticación de Windows[7][11][15]	Revisar `listen_addresses`, acotar `pg_hba.conf` y usar SCRAM-SHA-256[17][18]	Crear administrador, activar RBAC, firewall sobre el 27017 y TLS/SSL[24]

Las celdas marcadas como verificadas se apoyan en nuestra propia prueba. Son la versión 18.6, la codificación UTF8 con locale `es_ES.UTF-8` y la ubicación de los logs en `/var/log/postgresql/`. Las evidencias están en el Anexo A.

6 Conclusiones y recomendaciones

Qué hemos aprendido. La respuesta a nuestra pregunta es que los tres motores se parecen mucho en la mecánica de la instalación y se diferencian sobre todo en *dónde* colocan la responsabilidad de asegurarlos. Los tres fijan al instalar los mismos cuatro parámetros críticos, que son directorio de datos, puerto, codificación y administrador. Los tres escuchan en un puerto fijo y conocido. Y ninguno de los tres se expone a toda la red por defecto.

A partir de ahí se separan. SQL Server llega con las funciones peligrosas desactivadas de fábrica y deja una tarea manual clara y única, que es abrir el firewall. PostgreSQL reparte el control entre dos ficheros que actúan en capas distintas, `listen_addresses` y `pg_hba.conf`. MongoDB es el único que arranca sin control de acceso, de modo que su seguridad depende por completo de que el administrador complete el checklist antes de exponer el servicio.

Hay una segunda conclusión que no esperábamos al empezar. La codificación de caracteres ha dejado de ser un problema en el modelo documental. Mientras los dos relacionales obligan a elegir collation o locale en un momento prácticamente irreversible, MongoDB lo resuelve por diseño imponiendo UTF-8 en la propia especificación BSON.

Recomendaciones

- **SQL Server, cuando el entorno ya es Windows y hay directorio activo.** La integración con la autenticación de Windows evita gestionar contraseñas dentro del motor y el instalador guía la configuración segura. La condición: crear las reglas de firewall de forma explícita, mantener `sa` deshabilitada y asumir el coste de licencia salvo que Express cubra la carga.
- **PostgreSQL, como opción general en Linux.** Es la que mejor relación ofrece entre control, coste y facilidad de despliegue, tanto en instalación nativa como en contenedor. La condición: revisar siempre los dos cerrojos juntos, escucha y autenticación, y no dar el servicio por seguro habiendo tocado solo uno.
- **MongoDB, cuando el modelo de datos lo justifique.** Su instalación es la más sencilla de las tres, pero exige un procedimiento de aseguramiento posterior que no es negociable. La condición: verificar el soporte de AVX antes de provisionar la máquina y no mover `bindIp` de `localhost` hasta haber creado el administrador y activado `security.authorization`.
- **Para cualquiera de los tres.** Recomendamos tratar la instalación como un procedimiento escrito y repetible en lugar de como una secuencia de clics: instalación desatendida en SQL Server, script o Dockerfile en PostgreSQL, fichero de configuración versionado en MongoDB. Es la única forma de garantizar que la misma configuración segura se aplica en todos los servidores.

Límites del estudio

El estudio se apoya sobre todo en documentación oficial y solo uno de los tres motores se ha verificado en la práctica, en una única distribución y sobre máquina virtual. No hemos medido rendimiento, consumo de recursos ni comportamiento bajo carga. Tampoco hemos analizado el ciclo de vida posterior a la instalación, es decir, actualizaciones, copias de seguridad y alta disponibilidad, que sería la continuación natural de esta línea. Y no hemos entrado en los despliegues gestionados en la nube, donde parte de las decisiones de aseguramiento que aquí describimos las toma el proveedor.

— Referencias

[1] IBM. *¿Qué es un sistema de gestión de bases de datos (DBMS)?* 2026. ibm.com/es-es/topics/database-management-systems · Consulta: 29/09/2026.

[2] Oracle. *¿Qué es una base de datos?* 2026. oracle.com/es/database/what-is-database/ · Consulta: 29/09/2026.

[3] Microsoft (Microsoft Learn). *SQL Server 2025: Hardware and Software Requirements.* SQL Server 2025 (17.x). learn.microsoft.com/en-us/sql/sql-server/install/hardware-and-software-requirements-for-installing-sql-server-2025 · Consulta: 29/09/2026.

[4] Microsoft (Microsoft Learn). *Install SQL Server from the Installation Wizard (Setup).* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/install-windows/install-sql-server-from-the-installation-wizard-setup · Consulta: 29/09/2026.

[5] Microsoft (Microsoft Learn). *Install, Configure, or Uninstall SQL Server on Windows from the Command Prompt.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/install-windows/install-sql-server-from-the-command-prompt · Consulta: 29/09/2026.

[6] Microsoft (Microsoft Learn). *File Locations for Default and Named Instances of SQL Server.* SQL Server 2025 (17.x). learn.microsoft.com/en-us/sql/sql-server/install/file-locations-for-default-and-named-instances-of-sql-server · Consulta: 29/09/2026.

[7] Microsoft (Microsoft Learn). *Configure the Windows Firewall to Allow SQL Server Access.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/sql-server/install/configure-the-windows-firewall-to-allow-sql-server-access · Consulta: 29/09/2026.

[8] Microsoft (Microsoft Learn). *Default SQL Server Network Protocol Configuration.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/configure-windows/default-sql-server-network-protocol-configuration · Consulta: 29/09/2026.

[9] Microsoft (Microsoft Learn). *Configure Windows Service Accounts and Permissions.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/configure-windows/configure-windows-service-accounts-and-permissions · Consulta: 29/09/2026.

[10] Microsoft (Microsoft Learn). *Change Server Authentication Mode.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/configure-windows/change-server-authentication-mode · Consulta: 29/09/2026.

[11] Microsoft (Microsoft Learn). *Security Considerations for a SQL Server Installation.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/sql-server/install/security-considerations-for-a-sql-server-installation · Consulta: 29/09/2026.

[12] Microsoft (Microsoft Learn). *xp_cmdshell Server Configuration Option.* SQL Server 2017–2025. learn.microsoft.com/en-us/sql/database-engine/configure-windows/xp-cmdshell-server-configuration-option · Consulta: 29/09/2026.

[13] Microsoft (Microsoft Learn). *Collation and Unicode Support.* SQL Server 2019–2025. learn.microsoft.com/en-us/sql/relational-databases/collations/collation-and-unicode-support · Consulta: 29/09/2026.

[14] Microsoft (Microsoft Learn). *Ubuntu: Install SQL Server on Linux.* SQL Server 2025 (17.x). learn.microsoft.com/en-us/sql/linux/quickstart-install-connect-ubuntu · Consulta: 29/09/2026.

[15] Center for Internet Security (CIS). *CIS Microsoft SQL Server Benchmarks.* v1.0.0. cisecurity.org/benchmark/microsoft_sql_server · Consulta: 29/09/2026.

[16] The PostgreSQL Global Development Group. *PostgreSQL Documentation — Part III. Server Administration.* PostgreSQL 18 (2026). postgresql.org/docs/current/admin.html · Consulta: 29/09/2026.

[17] The PostgreSQL Global Development Group. *PostgreSQL Documentation — Chapter 20. Client Authentication.* PostgreSQL 18 (2026). postgresql.org/docs/current/client-authentication.html · Consulta: 29/09/2026.

[18] The PostgreSQL Global Development Group. *PostgreSQL Documentation — 19.3. Connections and Authentication.* PostgreSQL 18 (2026). postgresql.org/docs/current/runtime-config-connection.html · Consulta: 07/10/2026.

[19] Docker Official Images / PostgreSQL. *PostgreSQL — Official Image Documentation.* 2026. hub.docker.com/_/postgres · Consulta: 29/09/2026.

[20] MongoDB, Inc. *Install MongoDB Community Edition on Ubuntu.* MongoDB 8.0. mongodb.com/docs/manual/tutorial/install-mongodb-on-ubuntu/ · Consulta: 29/09/2026.

[21] MongoDB, Inc. *Production Notes for Self-Managed Deployments (Hardware Considerations).* MongoDB 8.0. mongodb.com/docs/manual/administration/production-notes/ · Consulta: 02/10/2026.

[22] MongoDB, Inc. *Self-Managed Configuration File Options.* MongoDB 8.0. mongodb.com/docs/manual/reference/configuration-options/ · Consulta: 29/09/2026.

[23] MongoDB, Inc. *BSON Types.* MongoDB 8.0. mongodb.com/docs/manual/reference/bson-types/ · Consulta: 02/10/2026.

[24] MongoDB, Inc. *Security Checklist.* MongoDB 8.0. mongodb.com/docs/manual/administration/security-checklist/ · Consulta: 02/10/2026.

ANEXO A

A Evidencias de la verificación práctica

Entorno de la prueba: UbuntuEscritorio26 sobre máquina virtual de Oracle VirtualBox, instalación por línea de comandos, realizada el **2 de octubre de 2026**. Versión obtenida: PostgreSQL 18.6 (Ubuntu 18.6-0ubuntu0.26.04.1).

```
pedro@pedro-seguridad-VirtualBox:~$ sudo apt install postgresql postgresql-contrib -y
Nota, seleccionando «postgresql» en lugar de «postgresql-contrib»
Instalando:
  postgresql

Instalando dependencias:
  libcommon-sense-perl  libpq5                  postgresql-18-jit        postgresql-common
  libjson-perl          libtypes-serialiser-perl  postgresql-client-18
  libjson-xs-perl       postgresql-18           postgresql-client-common

Paquetes sugeridos:
  libpq-oauth  postgresql-doc  postgresql-doc-18

Resumen:
  Actualizando: 0, Instalando 11, Eliminando: 0, no actualizando: 5
  Tamaño de la descarga: 18,3 MB
  Espacio necesario: 52,3 MB / 34,4 GB disponible

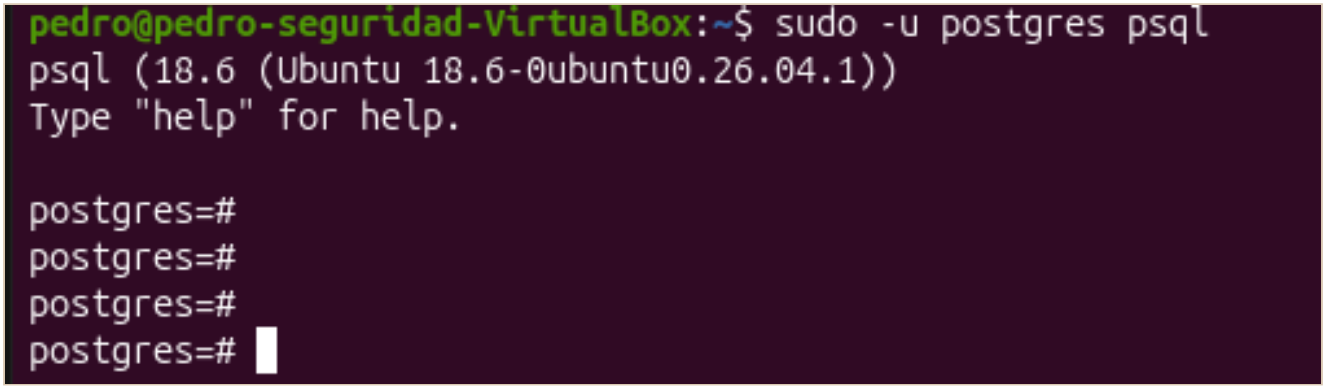
Des:1 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 libjson-perl all 4.10000-1 [81,9 kB]
Des:2 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 postgresql-client-common all 290ubuntu1 [49,4 k
B]
Des:3 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 postgresql-common all 290ubuntu1 [101 kB]
Des:4 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 libcommon-sense-perl amd64 3.75-3build5 [20,5 k
B]
Des:5 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 libtypes-serialiser-perl all 1.01-1 [11,6 kB]
Des:6 http://es.archive.ubuntu.com/ubuntu resolute/main amd64 libjson-xs-perl amd64 4.040-1 [84,4 kB]
Des:7 http://es.archive.ubuntu.com/ubuntu resolute-updates/main amd64 libpq5 amd64 18.6-0ubuntu0.26.04.1 [163
 kB]
Des:8 http://es.archive.ubuntu.com/ubuntu resolute-updates/main amd64 postgresql-client-18 amd64 18.6-0ubuntu
```

A.1 · Instalación del motor. Salida de `sudo apt install postgresql postgresql-contrib -y` tras actualizar los repositorios. Se instalan PostgreSQL 18 y las dependencias de servidor y cliente (`postgresql-18`, `postgresql-client-18`, `postgresql-common`). Confirma el método de instalación descrito en 4.2.

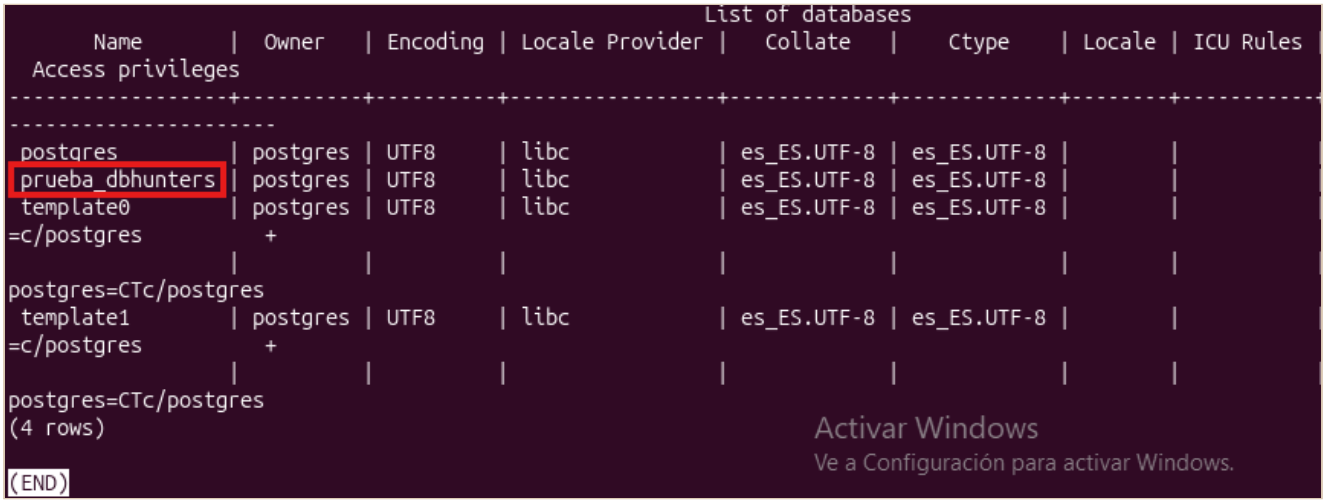
```
pedro@pedro-seguridad-VirtualBox:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
     Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
     Active: active (exited) since Fri 2026-10-02 12:33:56 CEST; 1min 49s ago
 Invocation: 0461389a6c2f4eab9814b4db8880fd4e
   Main PID: 21398 (code=exited, status=0/SUCCESS)
   Mem peak: 2M
        CPU: 9ms

oct 02 12:33:56 pedro-seguridad-VirtualBox systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
oct 02 12:33:56 pedro-seguridad-VirtualBox systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

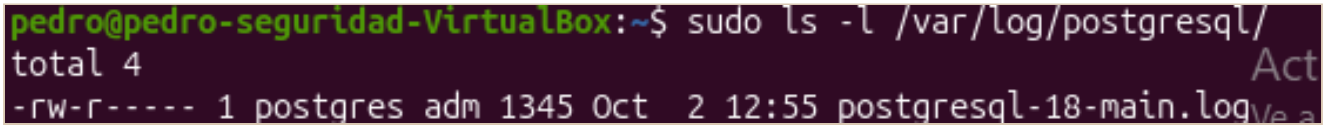
A.2 · Servicio arrancado. `sudo systemctl status postgresql` muestra la unidad cargada y *enabled*, con estado `active (exited)`. La unidad general es un metaservicio, así que ese estado es el esperado. La conexión posterior confirma que el servidor está operativo.



A.3 · Conexión al SGBD. `sudo -u postgres psql` abre el prompt `postgres=#`. Evidencia del usuario administrador inicial descrito en 4.3 y de su correspondencia con un usuario del sistema en la instalación nativa.



A.4 · Consulta al diccionario de datos. Resultado de `\l` después de ejecutar `CREATE DATABASE prueba_dbhunters;`. El catálogo refleja el cambio y pasa a listar cuatro bases de datos. En la columna *Encoding* se lee **UTF8** y en *Collate/Ctype* el locale **es_ES.UTF-8**. Este es el dato que usamos en la comparativa como verificación propia, no como afirmación del fabricante.



A.5 · Localización de los logs. `sudo ls -l /var/log/postgresql/` devuelve el fichero `postgresql-18-main.log`, confirmando la ruta de los registros de esta instalación.

Conclusión de la prueba. PostgreSQL se instala por comandos en UbuntuEscritorio26 y deja el servicio habilitado. Admite conexión con `psql` usando el usuario `postgres`, permite consultar el catálogo y crear bases de datos, y escribe sus registros en `/var/log/postgresql/`. En todo lo que pudimos comprobar, el resultado coincide con la documentación oficial.

ANEXO B

B Declaración de uso de IA y registro Scrum

B.1 Uso de herramientas de IA

USO	PARA QUÉ SE HA EMPLEADO	CÓMO SE HA VERIFICADO
Búsqueda	Localizar las páginas concretas de la documentación oficial de los tres fabricantes	Cada URL se ha abierto y leído directamente; la lista de referencias recoge la fecha de consulta
Resumen	Sintetizar apartados extensos de las guías de instalación y seguridad	Todo dato técnico se ha comprobado en la fuente antes de incorporarlo
Traducción	Traducir fragmentos de documentación en inglés	Contraste con la terminología de la unidad
Revisión y maquetación	Revisar la redacción y dar formato al informe final	Revisión del contenido por el equipo antes de la entrega

No hemos incorporado al informe ningún texto generado sin verificarlo antes en la fuente oficial o en la máquina de pruebas. Los resultados de la verificación práctica salen íntegramente de la ejecución real descrita en el Anexo A.

B.2 Resumen del proceso Scrum

SPRINT	SCRUM MASTER	HISTORIAS COMPLETADAS	VELOCIDAD
Sprint 1 25–30 sept	Daniel Madrigal	HU1 marco teórico · HU2 fuentes fiables · HU3 criterios de comparación	100 pts
Sprint 2 30 sept–2 oct	Pedro Hosannah	HU4 SQL Server · HU5 PostgreSQL · HU6 MongoDB · HU7 verificación práctica · HU8 comparativa	100 pts
Sprint 3 2–7 oct	Antonio Moga	HU9 informe final · HU10 presentación y defensa	100 pts

El Product Owner, Raúl Albiol, se mantuvo durante los tres sprints. El rol de Scrum Master rotó por acuerdo interno del equipo, que quedó registrado en el acta de constitución (Anexo A del Google Sheets). Las dos retrospectivas dieron lugar a acciones concretas que aplicamos en el sprint siguiente. Tras la Review 1 reorganizamos los anexos y entregables y ajustamos la estimación a 100 puntos por sprint. Tras la Review 2 acordamos entregar los tres documentos del proyecto juntos y dejar constancia del rol de cada miembro en cada sprint. El tablero, las actas de las dailies y los anexos A a I están en el Google Sheets del equipo, que se entrega junto a este informe y a la Guía de trabajo y organización.